

Email Spam Detection

This project implements a machine learning-based email spam detection system using various classification models, including Naive Bayes, Logistic Regression, Random Forest, and Support Vector Machine (SVM). The dataset contains emails labeled as "spam" or "ham" (non-spam), and the system aims to classify them accordingly.

Prerequisites

Before running the code, ensure you have the following installed:

1. **Python 3.x**
2. **Required Python Libraries:**
 - pandas
 - numpy
 - matplotlib
 - seaborn
 - wordcloud
 - scikit-learn
 - scipy

You can install the necessary libraries using pip:

```
pip install pandas numpy matplotlib seaborn wordcloud scikit-learn scipy
```

Dataset

The dataset is in CSV format with the name spam_ham_dataset3.csv. It should have the following columns:

- Unnamed: 0: An index column (to be removed during preprocessing).
- label: The label for the email, either spam or ham.
- text: The content of the email in text format.
- label_num: The numerical encoding of the label (spam: 1, ham: 0).

Steps to Run the Code

1. Load the Dataset

The dataset is loaded using pandas:

```
data = pd.read.csv("spam_ham_dataset3.csv")
```

Data Preprocessing

- The dataset is cleaned by removing unwanted columns and handling any missing values.

- Text data is cleaned by removing non-alphabetic characters and converting all text to lowercase.
- New features such as email_length (the length of each email) are created.

Code:

```
data['text'] = data['text'].apply(lambda x: re.sub(r'^a-zA-Z\s', '', x.lower())) # Text cleaning
```

Exploratory Data Analysis (EDA)

The following visualizations are created to better understand the dataset:

- **Distribution of Spam vs Ham** using countplot.
- **Email Length Analysis** using histplot.
- **Word Clouds** for spam and ham emails to visualize common words.

Code:

```
sns.countplot(x='label',data=data)
sns.histplot(data=data,x="email_length",hue="label",multiple="stack")
```

Feature Extraction and Scaling

- Text data is vectorized using TfidfVectorizer, which converts text into numerical features.
- A numeric feature, label_num, is scaled using MinMaxScaler.
- Both features are combined using scipy.sparse.hstack.

Code :

```
vectorizer = TfidfVectorizer()
X_text = vectorizer.fit_transform(data['text'])
scaler = MinMaxScaler()
X_numeric = scaler.fit_transform(data[['label_num']])
X_combined =.hstack((X_text, X_numeric))
```

Train-Test Split

The data is split into training and testing sets using train_test_split:

Code :

```
X_train, X_test, y_train, y_test = train_test_split(X_combined, y, test_size=0.2, random_state=42)
```

1. Model Training and Evaluation

Four models are trained and evaluated:

- **Naive Bayes** using MultinomialNB.

- **Logistic Regression** using LogisticRegression.
 - **Random Forest** using RandomForestClassifier.
 - **Support Vector Machine (SVM)** using SVC.
2. For each model, the following steps are performed:
- **Hyperparameter Tuning** using GridSearchCV.
 - **Model Evaluation** using classification_report and confusion_matrix.

Code :

Example for naive bayes model

```
nb_model = MultinomialNB()
nb_grid = GridSearchCV(nb_model, nb_param_grid, scoring='f1_weighted', cv=5,
n_jobs=-1)
```

Confusion Matrices and Classification Reports

After training and prediction, the classification reports and confusion matrices for each model are displayed. This gives a clear indication of each model's performance.

Code :

```
print(classification_report(y_test, y_pred_nb))
print(confusion_matrix(y_test, y_pred_nb))
```

Data Visualization

The project also includes visualizations for:

- **Spam vs Ham Distribution:** A bar plot that shows the proportion of spam and ham emails in the dataset.
- **Email Length Distribution:** A histogram that compares the lengths of emails in both spam and ham categories.
- **Word Clouds:** Word clouds visualizing the most frequent words in spam and ham emails.

Performance Metrics

For each model, the following performance metrics are calculated:

- **Accuracy**
- **Precision**
- **Recall**
- **F1-Score**

The models are evaluated with the F1-score as the primary metric.

Next Steps

- **Address Class Imbalance:** Use techniques such as oversampling or undersampling to handle class imbalance issues more effectively.

- **Deploy the Model:** Consider deploying the trained models to detect spam emails in real-time applications.

Output

After running the code, you should see the following output:

1. **Distribution of Spam vs Ham:** A count plot showing the number of spam and ham emails.
2. **Email Length Analysis:** A histogram that shows how the email length varies between spam and ham.
3. **Word Clouds:** Two word clouds showing the most frequent words in spam and ham emails.
4. **Model Evaluation:** Classification reports and confusion matrices for each model.